

Processors

Processors apply business logic to the data stream between readers and writers. This guide focuses on usage; detailed method signatures belong in the API docs.

When to use a processor

Use a processor when you need to transform, filter, or enrich data before writing it to the target system.

Common usage pattern

1. Implement a custom processor by extending the base class.
2. Instantiate the processor and pass it into `Task`.
3. Execute the task with `TaskExecutor`.

```
from datasyncx.core.task import Task
from datasyncx.core.task_executor import TaskExecutor
from datasyncx import FileReader, SqlServerWriter
from datasyncx.core.base_component import Processor
from pandas import DataFrame

class MyProcessor(Processor):
    def process(self, df: DataFrame, *args, **kwargs) -> DataFrame:
        # custom business logic
        return df

reader = FileReader(...)
writer = SqlServerWriter(...)
processor = MyProcessor()

task = Task(reader=reader, writer=writer, processor=processor, receiver="user@intel.com")
TaskExecutor().TaskRun(task=task)
```

Multiple processors

`Task` accepts either a single processor instance or a list of processor instances. When you pass a list, DataSyncX runs them in order and feeds the output of each processor into the next one.

```

from datasyncx.core.task import Task
from datasyncx.core.task_executor import TaskExecutor
from datasyncx import FileReader, SqlServerWriter
from datasyncx.processors.dataframe_processor import (
    DropColumnsProcessor,
    RenameColumnsProcessor,
)

reader = FileReader(...)
writer = SqlServerWriter(...)
processors = [
    RenameColumnsProcessor(column_mapping={"old_name": "new_name"}),
    DropColumnsProcessor(columns_to_drop=["debug_column"]),
]

task = Task(
    reader=reader,
    writer=writer,
    processor=processors,
    receiver="user@intel.com",
)
TaskExecutor().TaskRun(task=task)

```

If you are using `job_spec` or `demo_factory`, the `processor` field can also be either a single object or a list of objects.

```

processor:
  - type: rename_columns
    params:
      column_mapping:
        old_name: new_name
  - type: drop_columns
    params:
      columns_to_drop:
        - debug_column

```

Copy-ready custom processor template

The built-in dataframe processors are useful examples, but for most projects the best copy-and-modify starting point is a tiny custom processor like this:

```

from pandas import DataFrame

from datasyncx.core.base_component import Processor

class CustomDataFrameProcessor(Processor):
    def __init__(self, required_columns=None):
        super().__init__()
        self.required_columns = list(required_columns or [])

    def process(self, df: DataFrame, *args, **kwargs) -> DataFrame:
        missing_columns = [
            column for column in self.required_columns if column not in df.columns
        ]
        if missing_columns:
            raise ValueError(
                "Missing required columns for CustomDataFrameProcessor: "
                f"{missing_columns}"
            )

        result = df.copy()
        # Put your dataframe transformation logic here.
        return result

```

For a full direct-demo script that embeds a custom reader, processor, and writer in one file, see `src/datasyncx/demo/MyReader_to_MyWriter.py`.

Notes

- Keep processors side-effect free when possible.
- Prefer small, composable transforms for complex logic.
- A processor list works best when each step has one clear responsibility.
- For a full direct-demo script that embeds a custom processor, see

`src/datasyncx/demo/MyReader_to_MyWriter.py`.